

2026 年 4 月 3 日架构师网络课程纪要

一、本次课程总结

【课堂主题】

《案例特训（架构篇）》

【重点内容】



数据访问层设计



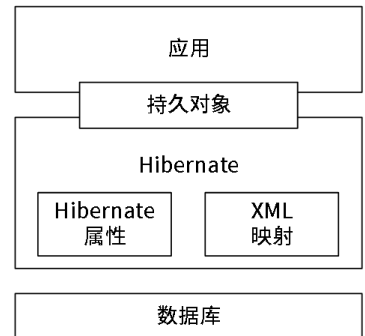
ORM（Object Relational Mapping）：对象与关系数据之间的映射。

映射关系表

面向对象	关系数据库
类（class）	数据库的表（table）
对象（object）	记录（record，行数据）
对象的属性（attribute）	字段（field）

实现技术对比表

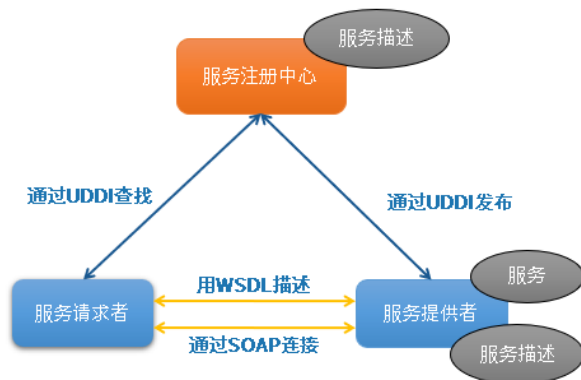
维度	Hibernate	MyBatis（iBatis）
简单对比	强大，复杂，间接，SQL无关	小巧，简单，直接，SQL相关
可移植性	好（不关心具体数据库）	差（根据数据库SQL编写）
复杂多表关联	不支持	支持



Hibernate 架构图



Web Service (WEB服务)



【UDDI】是 Web 服务集成的一个体系框架，包含了服务描述与发现的标准规范。

【WSDL】服务描述语言，三个基本属性（服务做些什么 / 如何访问服务 / 服务位于何处）。

【SOAP】基于 XML 的协议，在分布式环境中交换信息。4 个部分：SOAP 封装（消息内容，谁发的，谁接收）、SOAP 编码规则（数据类型实例）、SOAP RPC（远程过程调用和应答协定）、SOAP 绑定（使用底层协议交换信息）。

REST 的 5 个原则

- (1) 网络上的所有事物都被抽象为资源。
- (2) 每个资源对应一个唯一的资源标识（反过来不成立）。
- (3) 通过通用的连接件接口对资源进行操作。
- (4) 对资源的各种操作不会改变资源标识。
- (5) 所有的操作都是无状态的。



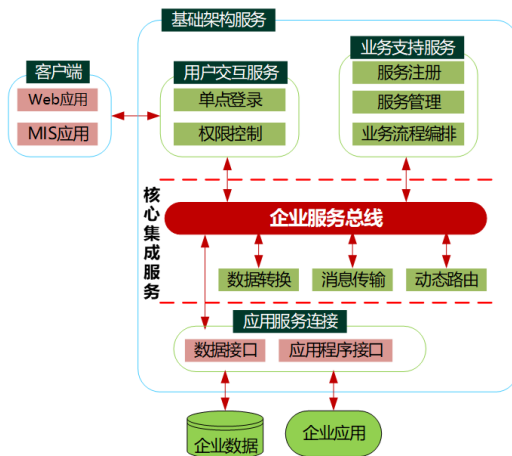
REST (表述性状态转移)

REST API

命令	用途
GET	从指定的资源请求数据，只进行数据检索，不进行其他操作。
POST	将数据发送到服务器进行创建，通常用于上传文件或提交表单。
PUT	更新目标资源的所有当前表示，使用上传的内容进行替换。
DELETE	删除指定的资源。
HEAD	与 GET 方法类似，但只传输状态行和头部信息。
PATCH	对资源进行部分修改。



ESB (企业服务总线)



服务请求者与服务提供者之间解耦

【ESB的核心功能】

- ✓ 提供位置透明性的消息路由和寻址服务
- ✓ 提供服务注册和命名的管理功能
- ✓ 支持多种消息传递范型
- ✓ 支持多种可以广泛使用的传输协议
- ✓ 支持多种数据格式及其相互转换
- ✓ 提供日志和监控功能



微服务



微服务的优点解读

优点	解读
【复杂应用解耦】	小服务（且专注于做一件事情） 化整为零，易于小团队开发
【独立】	独立开发 独立测试及独立部署（简单部署） 独立运行（每个服务独立在其独立进程中）
【技术选型灵活】	支持异构（如：每个服务使用不同数据库）
【容错】	故障被隔离在单个服务中，通过重试、平稳退化等机制实现应用层容错
【松耦合，易扩展】	可根据需求独立扩展

面临的问题与挑战【缺点】

- 分布式环境下的数据一致性【更复杂】
- 测试的复杂性【服务间依赖测试】
- 运维的复杂性



微服务与SOA



微服务	SOA
能拆分的就拆分	是整体的，服务能放一起的都放一起
纵向业务划分	是水平分多层
由单一组织负责	按层级划分不同部门的组织负责
细粒度	粗粒度
两句话可以解释明白	几百字只相当于SOA的目录
独立的子公司	类似大公司里面划分了一些业务单元（BU）
组件小	存在较复杂的组件
业务逻辑存在于每一个服务中	业务逻辑横跨多个业务领域
使用轻量级的通信方式，如HTTP	企业服务总线（ESB）充当了服务之间通信的角色



云计算概念/优点/分类



【云计算】是集合了大量计算设备和资源，对用户屏蔽底层差异的分布式处理架构，其用户与提供实际服务的计算资源是相分离的。

【云计算优点】超大规模、虚拟化、高可靠性、高可伸缩性、按需服务、成本低【前期投入低、综合使用成本也低】

云计算【按服务类型分类】

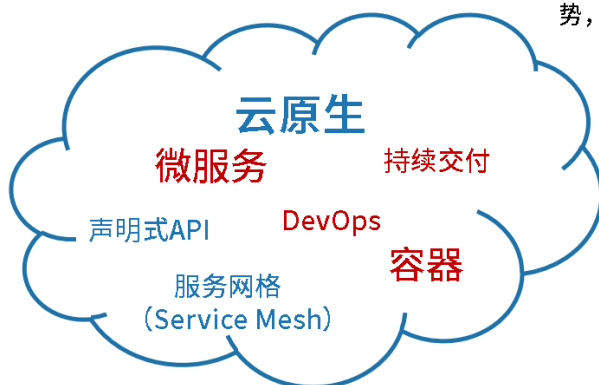
SaaS【软件即服务】	基于多租户技术实现，直接提供应用程序
PaaS【平台即服务】	虚拟中间件服务器、运行环境和操作系统
IaaS【基础设施即服务】	包括服务器、存储和网络等服务



云原生架构



【云原生 (Cloud Native)】一开始就是基于云环境、专门为云端特性而设计，可充分利用和发挥云平台的弹性+分布式优势，最大化释放云计算生产力。



↓【本质】
Oncloud ⇌ Incloud



云原生架构



【云原生架构设计原则】

- ✓ 服务化原则：使用微服务
- ✓ 弹性原则：可根据业务变化自动伸缩
- ✓ 可观测原则：通过日志、链路跟踪和度量
- ✓ 韧性原则：面对异常的抵御能力
- ✓ 所有过程自动化原则：自动化交付工具
- ✓ 零信任原则：默认不信任网络内部和外部的任何人/设备/系统
- ✓ 架构持续演进原则：业务高速迭代情况下的架构与业务平衡



云原生架构



【云原生架构模式】

- 1、服务化架构模式：典型代表【微服务】，服务拆分使维护压力大增。
- 2、Mesh化架构模式：把中间件框架（RPC、缓存、异步消息）从业务进程中分离，由Mesh进程完成。
- 3、Serverless模式【无服务器计算】：非常适合于事件驱动的数据计算任务。
- 4、存储计算分离模式：各类暂态数据（如session）用云服务保存。
- 5、分布式事务模式：解决微服务模式中多数据源事务问题。
- 6、可观测架构：包括Logging、Tracing、Metrics三个方面。
- 7、事件驱动架构：本质上是一种应用/组件间的集成架构模式。



云原生架构



维度	Service Mesh (典型: Istio)	Serverless (典型: 函数计算 FC)
核心解决什么问题	解决“微服务之间怎么可靠、高性能、可观测地通信”	解决“ 开发者根本不想管服务器、扩容、运维 ”
你眼里看到的形态	一堆容器 (K8s Pod) + 一个隐形的“超级侧车” (envoy)	完全看不到服务器, 只有“函数”
部署单元	每个微服务一个 Pod + 一个 Envoy 侧车	一个业务功能 = 一个函数
谁来管扩容缩容	你自己写 HPA + 配置 Istio 流量治理	云厂商 自动、毫秒级 、精确到每次调用
谁来管重试/熔断/限流	Istio 统一治理, 全局策略, 一键下发	你要么自己写在函数里, 要么用 API 网关 + 函数组合实现
冷启动	没有 (容器常驻)	有 (50ms~3s), 高频链路不能忍
适合任务时长	常驻服务、长连接、WebSocket、gRPC 流	短任务 (<15分钟)、突发、异步
运维复杂度 (2025)	较高 (要管 K8s + Istio + 各种 CRD)	极低 (基本不管)
典型应用场景	学员服务、订单服务、支付服务、课程服务、直播服务……这几十个微服务之间互相调用	作业自动批改、直播结束生成回看、Excel 导入、推送通知、视频转码、数据报表

【视频回看地址】

希赛网目前提供了两种课程回看的方式:

方式一: 课程回放【课程结束后 20 分钟内可立即回看】

APP 观看路径: 我的->网络课堂

PC 观看路径: 学习中心->网络课堂

回看注意事项:

(1) 首先登录希赛网(<https://www.xisaiwang.com/>), 然后使用以上地址回看。

(2) 回看时如果需要拖放观看, 有两种方法进行视频内容定位: 方法一是将鼠标移至视频头像位置进行拖放; 方法二是直接点击播放界面下方的微缩图, 可直接定位到这一页 PPT 的讲解。

方式二: 老师端录屏回看【课程结束后 1-2 个工作日发布】

APP 观看路径: 我的->视频课程

PC 观看路径: 学习中心->视频课程

二、下次课程安排

主题: 《案例特训 (架构篇)》

时间: 2026 年 4 月 8 日【周三】19: 30 - 21: 30

预习要求:

回顾软件架构设计章节知识内容。

视频地址为：

<https://wangxiao.xisaiwang.com/nstu/sub/course/courseDetail/courseDetail?courseId=13147>

三、课后作业

阅读以下有关表现层设计方面的说明，在答题纸上回答问题 1 至问题 3。

【说明】

某商业银行欲开发一套个人银行系统，为用户提供常见的金融服务，包括转账、查询、存款变更和个人信息管理等功能。该软件除了业务需求外，还有一些特殊的表现层需求：

- (1) 根据用户级别的不同，界面和可用功能是不同的；
- (2) 支持 Web、Windows、手机 App 等多种不同类型的界面；
- (3) 考虑到将来功能的扩展，需要系统支持界面的定制以及动态生成等功能，以降低系统维护和新功能发布的成本。

经过对需求的讨论，该银行初步决定采用 MVC 模式设计该个人银行系统的表现层，采用 XML 作为 GUI 的描述语言，并应用 XML 的界面管理技术来实现灵活的界面配置、界面动态生成和界面定制。

问题内容：

【问题 1】（9 分）

MVC 模式强制性地将一个应用处理流程按照模型、视图、控制的方式进行分离，三者的协作关系如图 1 所示。

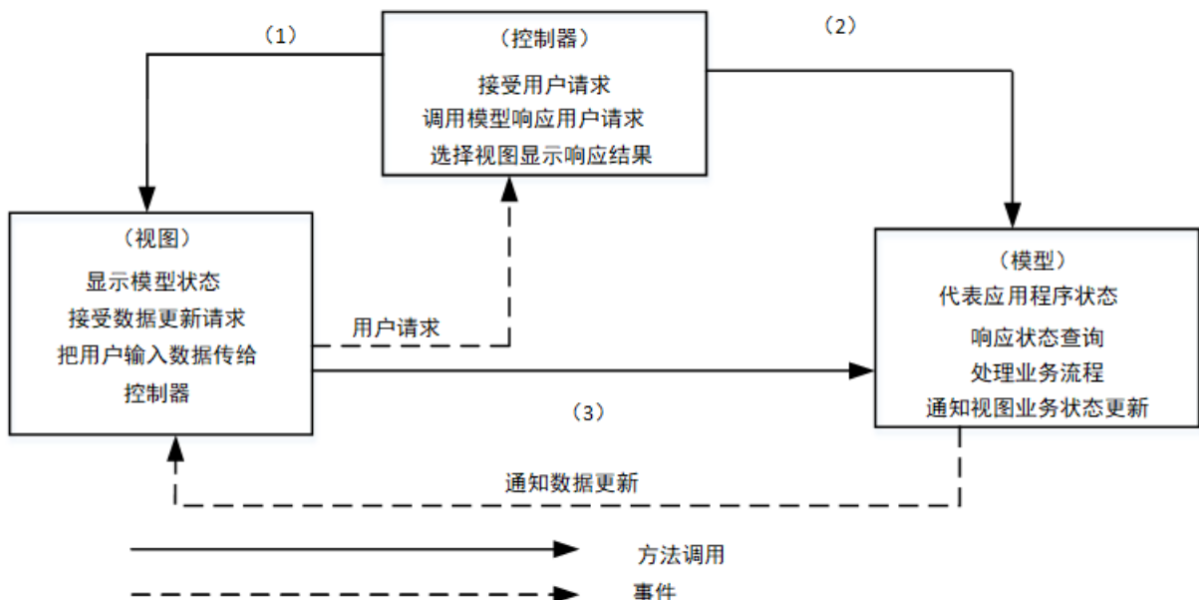


图 1 MVC 设计模式

请填写图 1 中的（1）～（3），并简要说明在该个人银行系统中采用 MVC 模式对界面设计的作用。

【问题 2】（4 分）

请从设计模式的角度，简要说明设计方案采用 XML 作为 GUI 描述语言的机制。

【问题 3】（12 分）

基于 XML 的界面管理技术可实现灵活的界面配置、界面动态生成和界面定制，其思路是用 XML 生成配置文件及界面所需的元数据，按不同需求生成界面元素及软件界面，其技术框图如图 2 所示。

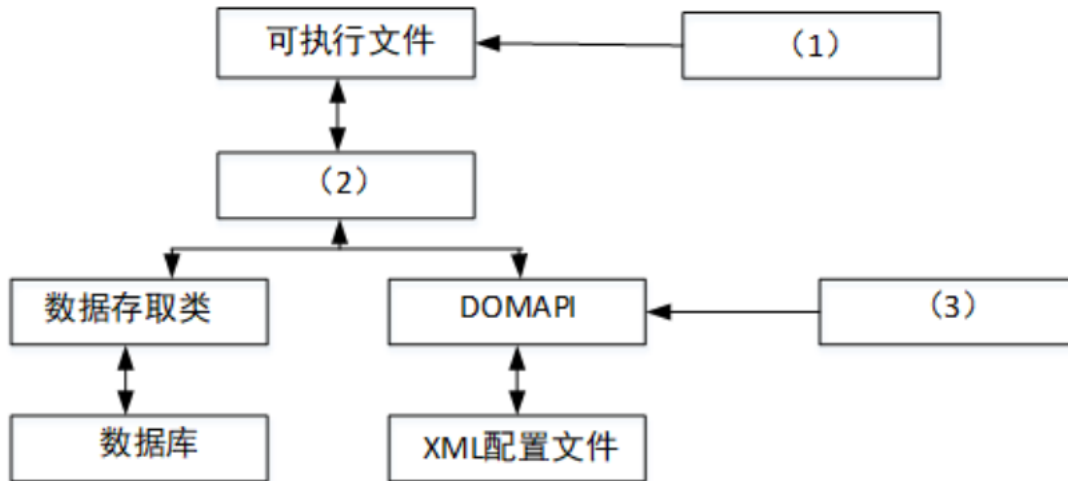


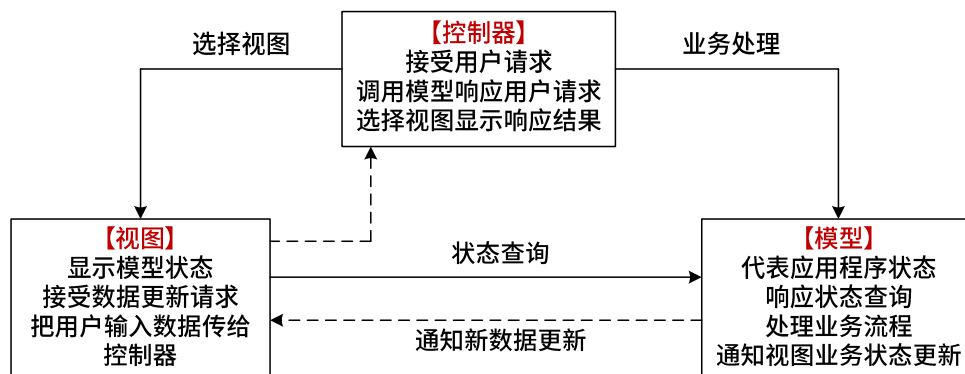
图 2 基于 XML 的界面管理技术框图

请将恰当的内容填入图 2 中的 (1) ~ (3)，并简要解释说明其含义。

答案：

【问题 1】

(1) 选择视图 (2) 业务处理 (3) 状态查询



【图例】
 方法调用
 事件

MVC 模式对该个人银行系统的作用：

- (1) 允许多种界面的扩展，视图的变更与增加，与模型无关；
- (2) 易于维护，控制器和视图随着模型的扩展而扩展，只要保持公共接口，控制器和视图的旧版本可以继续使用；

(3) 可支持功能强大的用户界面。

【问题 2】

从设计模式的角度来说，整个 XML 表现层解析的机制是一种策略模式。在调用显示 GUI 时，不是直接调用特定的表现技术的 API，而是装载 GUI 对应的 XML 配置文件，然后根据特定的表现技术的解析器解析 XML，得到 GUI 视图实例对象。这样，对于 GUI 开发人员来说，GUI 视图只需要维护一套 XML 文件即可。

【问题 3】

(1) 界面定制

(2) 界面动态生成

(3) 界面配置

界面配置是对用户界面的静态定义，通过读取配置文件的初始值对界面配置。由界面配置对软件功能进行裁剪、重组和扩充，以实现特殊需求。

界面定制是对用户界面的动态修改过程，在软件运行过程中，用户可按需求和使用习惯，对界面元素（如菜单、工具栏、键盘命令）的属性（如文字、图标、大小、位置等）进行修改。软件运行结束后，界面定制的结果被保存。

界面动态生成是系统通过 DOM API 读取 XML 配置文件的表示层信息（初始界面大小、位置等），通过数据存取类读取数据库中的数据层信息，运行时由界面元素动态生成界面。界面配置和定制模块在软件运行前后修改配置文件、更改界面内容。